Architecture hardware

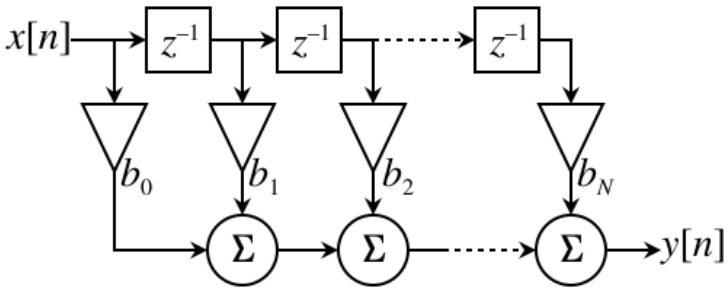
semestre printemps 2025 - 2026

Objectifs pédagogiques

Ce laboratoire a pour but de réaliser un filtre FIR combinatoire, séquentiel, et pipeliné.

Cahier des charges

Dans l'idée de réaliser un filtre FIR, il vous est demandé d'en réaliser un selon différentes architectures.
Un filtre à réponse impulsionnelle finie correspond au traitement illustré sur le schéma suivant :



Le flux de données passe au travers un registre à décalage et les $N+1$ (pour un filtre d'ordre N) dernières valeurs reçues servent à calculer la sortie, selon l'équation suivante :

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] + \cdots + b_N x[n-N]$$

Les valeurs des coefficients b_i définissent la fonction de transfert du filtre.
Dans ce laboratoire, `ORDER` définit l'ordre du filtre. Le nombre total de coefficients (et donc de taps) est `ORDER+1`.

Entrées/sorties

Les entrées/sorties de l'entité sont les suivantes :

```
entity fir_filter is
    generic (
        ORDER       : positive := 8;     -- Filter order
        DATASIZE    : positive := 16;    -- Input/output data width
        COEFFSIZE   : positive := 16;    -- Coefficient width
        COMMAPOS    : natural := 0;      -- Position of comma for computations
    );
    port (
        clk_i        : in  std_logic;
        rst_i        : in  std_logic;
        din_valid_i  : in  std_logic;
        din_i        : in  std_logic_vector(DATASIZE-1 downto 0);
        din_ready_o  : out std_logic;
        coeffs_i     : in  coeff_array(0 to ORDER);
        dout_valid_o : out std_logic;
        dout_o       : out std_logic_vector(DATASIZE-1 downto 0);
        dout_ready_i : in  std_logic;
    );
end entity fir_filter;
```

Le filtre prend en entrée un flux de données. Une donnée est consommée lorsque les deux signaux `din_valid_i` et `din_ready_o` sont actifs.
Le résultat est considéré valide lorsque `dout_valid_o` est actif, et est consommé lorsque `dout_valid_o` et `dout_ready_i` sont actifs simultanément.
Les paramètres génériques permettent de calibrer le filtre. Ils permettent de définir l'ordre du filtre, la taille des données, la taille des coefficients, et la position de la virgule des coefficients. Cette dernière permet de tronquer correctement les valeurs avant de les transmettre à la sortie.

Banc de test

Un banc de test vous est fourni. Il est exploitable pour les trois architectures. Utilisez les scripts *sim.do* afin d'avoir toujours la possiblité de simuler rapidement avec l'un ou l'autre des designs. Il est toutefois relativement simple, n'hésitez pas à le modifier.

Combinatoire

La première implémentation doit être purement combinatoire, à l'exception du registre permettant de stocker les valeurs précédentes.

Séquentiel

La version séquentielle doit traiter les données en essayant de minimiser les ressources nécessaires. Il s'agira donc clairement de n'avoir qu'un seul multiplicateur.

Pipeliné

La version pipelinée doit permettre d'obtenir une meilleure fréquence d'horloge que la version combinatoire. A vous de réfléchir à la manière de bien découper les calculs.

Synthèse

Nous n'allons pas intégrer les systèmes sur carte, mais sommes néanmoins intéressés par la synthèse. Pensez évidemment à faire des synthèses durant le développement afin de ne pas simuler du code non synthétisable. Aussi, nous serons intéressés à savoir combien de ressources sont nécessaires pour vos trois implémentations, pour la cible correspondant à la DE1SoC. Pour ces synthèses, une entité `fir_filter_synth` vous est fournie. Il ajoute des registres en sortie pour faciliter la mesure de fréquence max.

Travail à rendre

Vous devez rendre toutes les sources, ainsi qu'un petit rapport.
Votre rapport doit contenir les informations suivantes :

1. Présentation des choix que vous avez fait pour les trois designs
2. Schémas des trois implémentations (il est tout à fait possible de prendre une photo d'un dessin)
3. Résultat en synthèse des trois implémentations (logique + nombre de registres). Ne pas oublier la fréquence d'horloge.
4. Une analyse de ces synthèses, analysant les pour/contre des différents designs.
5. Une description claire et complète de l'usage de l'IA.